Malaviya National Institute of Technology Jaipur
Department of Computer Science and Engineering

Padding Oracle Attack on AES-CBC

Recovering plaintext without the key

Course	Cryptography Laboratory (22CPP307)
Assignment	Assignment 7 (Padding Oracle Attack)
Group	Group 10 (Even group number)
Team members	Rishi Saini (2024UCP1566), Nandini Verma (2024UCP1667)
Instructor	Dr. Meenakshi Tripathi
Submission date	15 October 2026
Language	Python 3 (PyCryptodome for AES only)

1. Understanding the attack

AES-CBC

CBC encrypts a message block by block: $C_i = AES_k(P_i \oplus C_{i-1})$ with $C_0 = IV$. Decryption is $P_i = AES_k^{-1}(C_i) \oplus C_{i-1}$. The previous ciphertext block is XOR-ed directly into the decrypted block.

PKCS#7 padding

AES works on 16-byte blocks, so a message is padded with N bytes each of value N ($1 \le N \le 16$); a message that already fills the last block receives a whole block of 0x10. On decryption the receiver checks that the last N bytes all equal N.

The padding oracle

If a system reveals - through an error message, status code or even timing - whether the padding of a decrypted ciphertext was valid, it acts as an **oracle** that answers one yes/no question per query. The attacker can choose the ciphertext, so the oracle leaks information about the intermediate value $AES^{-1}(C_i)$.

2. Implementation

File	Role
src/aes_cbc.py	AES-CBC encryption/decryption with PKCS#7 and the padding oracle (the key lives only inside this class)
src/padding_oracle_attack.py	Attack engine: block-by-block, byte-by-byte recovery using only oracle(iv, ciphertext) → True/False
src/main.py	Scenarios, analysis output (--auto) and interactive menu
testcases/test_padding_oracle.py	17 unit tests (single/multi block, exact-multiple lengths, random messages, no key access)

```
for each ciphertext block C_i with predecessor C_(i-1):          # for block 1 the predecessor is the IV
    I = [0]*16                                          # intermediate bytes AES_dec(C_i)
    for pos = 15 down to 0:                             # right to left
        pad = 16 - pos
        for g = 0 .. 255:
            forged = zeros;  forged[pos+1..15] = I[..] XOR pad;  forged[pos] = g
            if oracle(forged, C_i) is valid:            # padding accepted
                if pad == 1: re-test with forged[pos-1] changed (rules out a false 0x02 0x02 hit)
                I[pos] = g XOR pad;  break
    P_i = I XOR C_(i-1)
```

The AES key is never read by the attack: it only sees `oracle(iv, ciphertext) → bool`. The program is not tied to a particular plaintext - it works for any message length (tested on 15, 32, 58 and 137-byte messages).

3. Results

Scenario	Plaintext length	Blocks	Oracle queries	Worst case	Saving	Recovered
Group 10 Assigned Message	58 B	4	7,494	16,384	54.3 %	100 % exact
Short Message (Single Block)	15 B	1	1,844	4,096	55.0 %	100 % exact
Exactly Two Blocks (32 bytes)	32 B	3	6,301	12,288	48.7 %	100 % exact
Long Multi-Block Message	137 B	9	18,621	36,864	49.5 %	100 % exact

Assigned message (Group 10): `CryptoLabX Group 10 Secret Message for Padding Oracle Lab!` - recovered completely (58 bytes, 4 blocks).

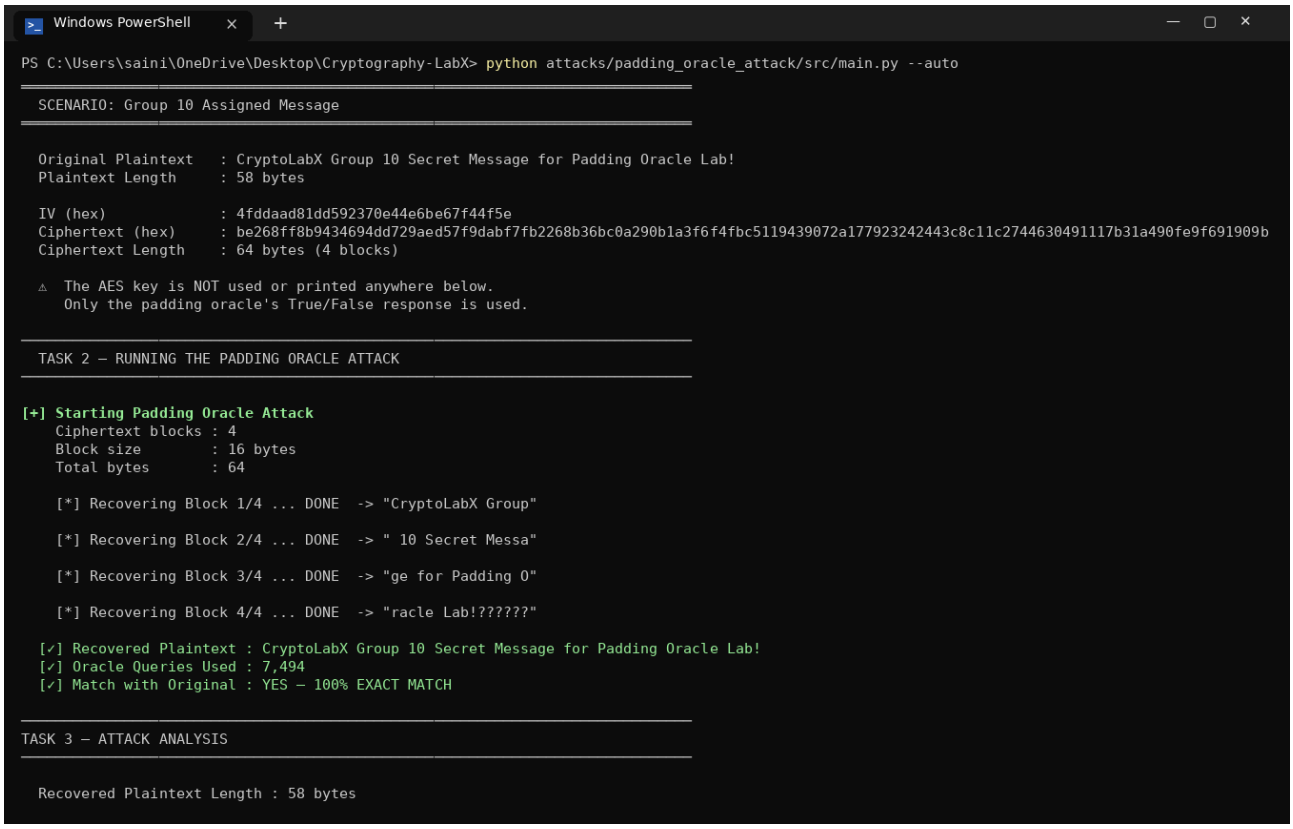


Figure 1 - Attack on the Group 10 message

4. Analysis

Query count. Each byte needs between 1 and 256 guesses (average ≈ 128), plus a few extra confirmation queries for the last byte of a block. For N blocks the worst case is 256 × 16 × N and the average about 128 × 16 × N. The measured counts in the table are close to the average, saving roughly 50 % against the worst case.

Why modifying C_{i-1} changes P_i. Because $P_i = AES^{-1}(C_i) \oplus C_{i-1}$ and $AES^{-1}(C_i)$ does not change when only C_{i-1} is edited, flipping a bit of byte j of C_{i-1} flips exactly the same bit of byte j of P_i. The attacker therefore controls the last bytes of the decrypted block and can force them to look like valid padding; the guess that makes the oracle say 'valid' reveals the hidden intermediate byte,

and XOR with the real C_{i-1} gives the plaintext byte.

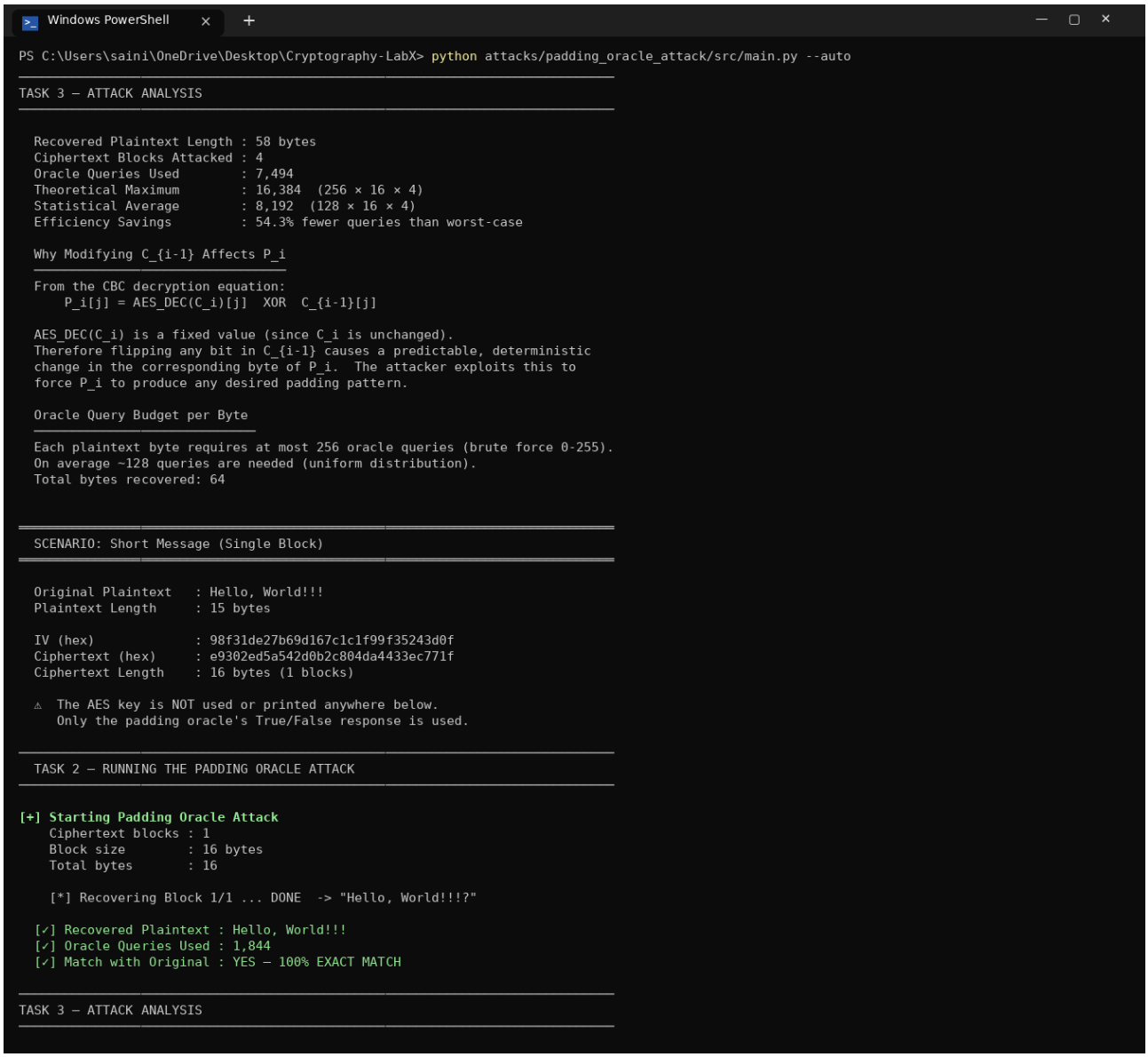


Figure 2 - Query analysis

5. Security recommendation

- **Use authenticated encryption** (AES-GCM or ChaCha20-Poly1305). Tampered ciphertexts are rejected before any padding exists.
- **Encrypt-then-MAC** (HMAC-SHA256 over IV + ciphertext) and verify the MAC *first* in constant time if CBC must be used.
- Return one generic error for every decryption failure; avoid distinguishable messages, status codes and timing differences.
- Use current protocols (TLS 1.3 has no CBC cipher suites) and rate-limit/monitor repeated decryption failures.

6. Test evidence

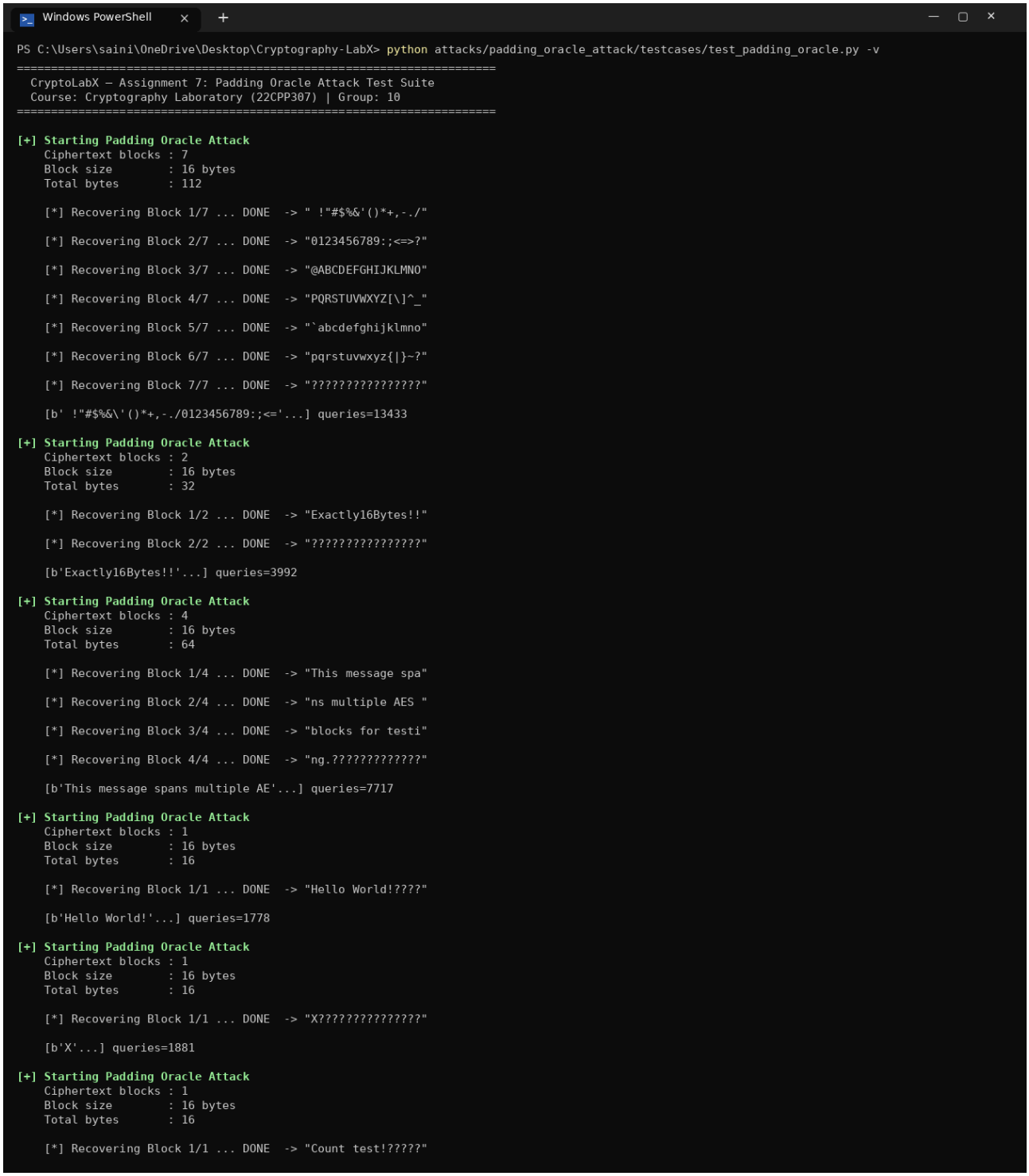


Figure 3 - 17 unit tests passing

7. Conclusion

A padding oracle turns a one-bit answer into complete plaintext recovery in a few thousand queries, without the key. The fix is cryptographic (authenticate before decrypting), not cosmetic.